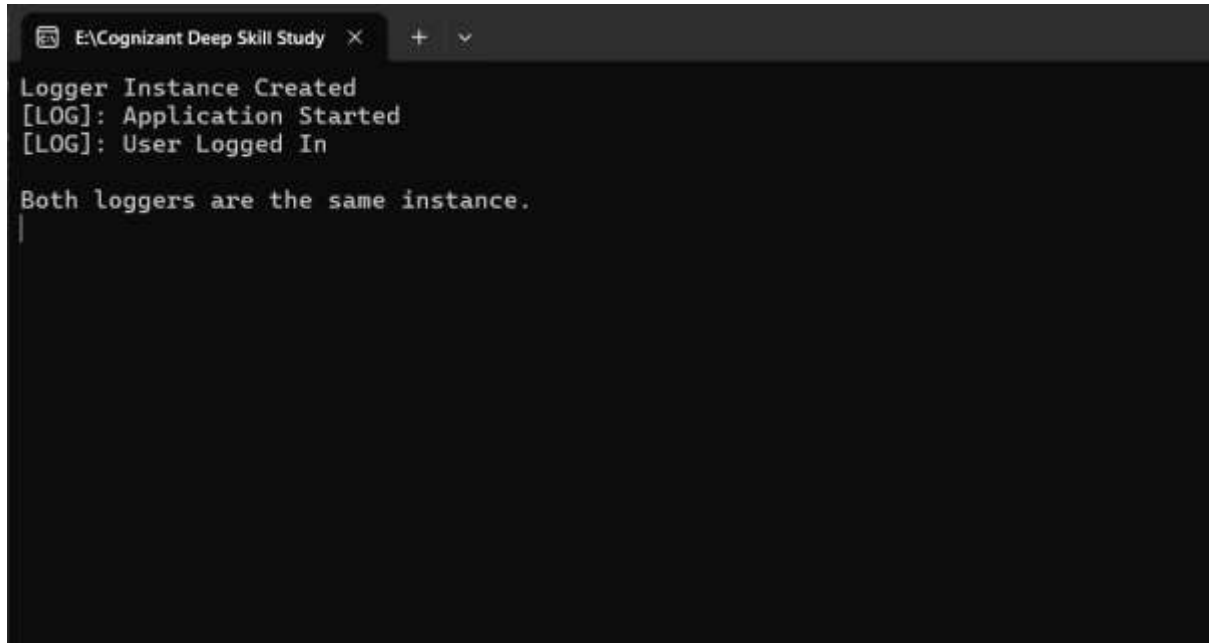


Design Patterns Exercise Outputs

Exercise 1: Implementing the Singleton Pattern

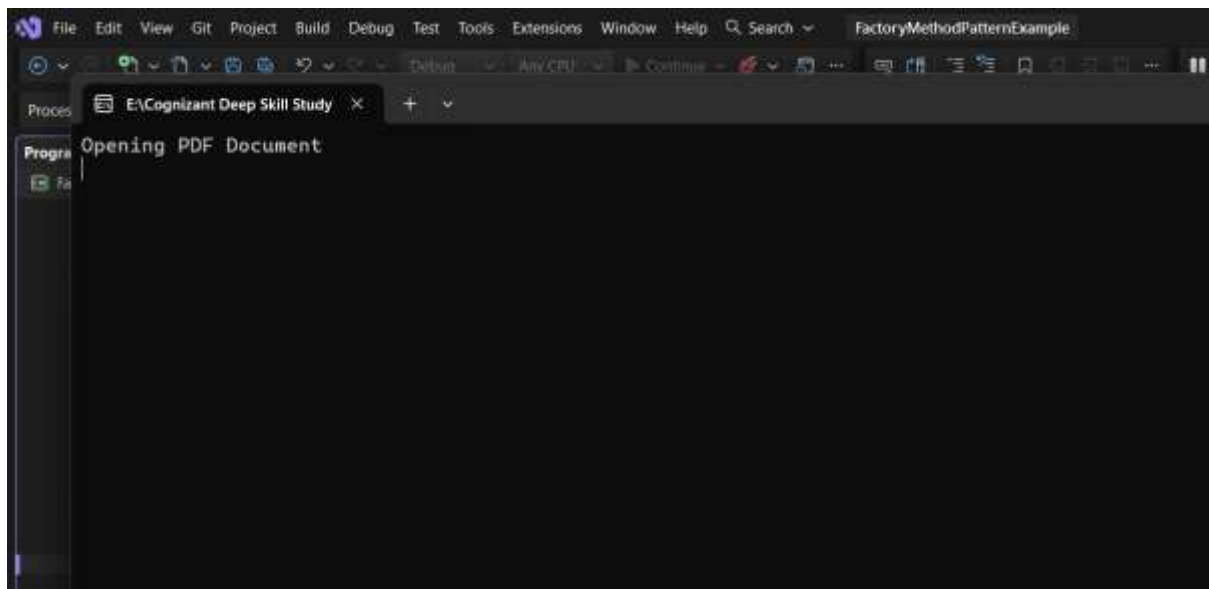


```
E:\Cognizant Deep Skill Study >
Logger Instance Created
[LOG]: Application Started
[LOG]: User Logged In

Both loggers are the same instance.
```

The screenshot shows a terminal window with a dark background. The title bar indicates the directory is 'E:\Cognizant Deep Skill Study'. The output text shows that a logger instance was created, followed by two log messages: '[LOG]: Application Started' and '[LOG]: User Logged In'. Finally, it states 'Both loggers are the same instance.', demonstrating the Singleton pattern where only one instance of the logger exists.

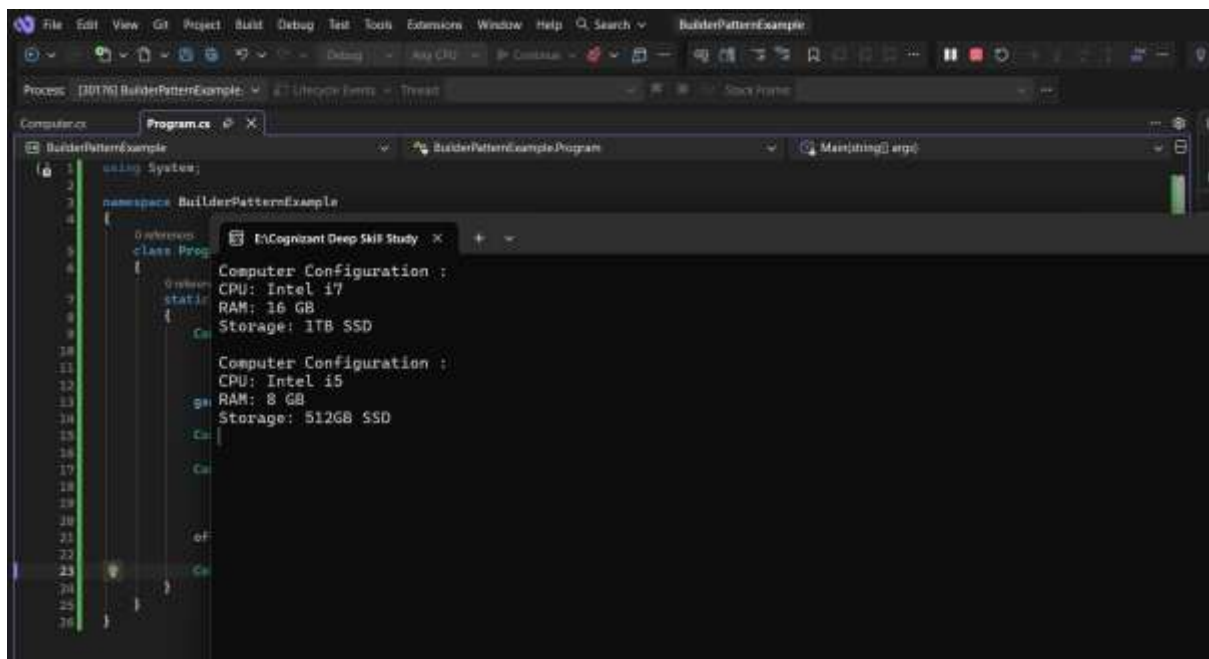
Exercise 2: Implementing the Factory Method Pattern



```
File Edit View Git Project Build Debug Test Tools Extensions Window Help Search FactoryMethodPatternExample
Debug Any CPU Continue
Process E:\Cognizant Deep Skill Study >
Opening PDF Document
```

The screenshot shows a Visual Studio Code window with a dark theme. The title bar includes the menu bar (File, Edit, View, Git, Project, Build, Debug, Test, Tools, Extensions, Window, Help) and the search bar. The active file is 'FactoryMethodPatternExample'. The output window shows the text 'Opening PDF Document', indicating the execution of a factory method that creates a new instance of a PDF document.

Exercise 3: Implementing the Builder Pattern

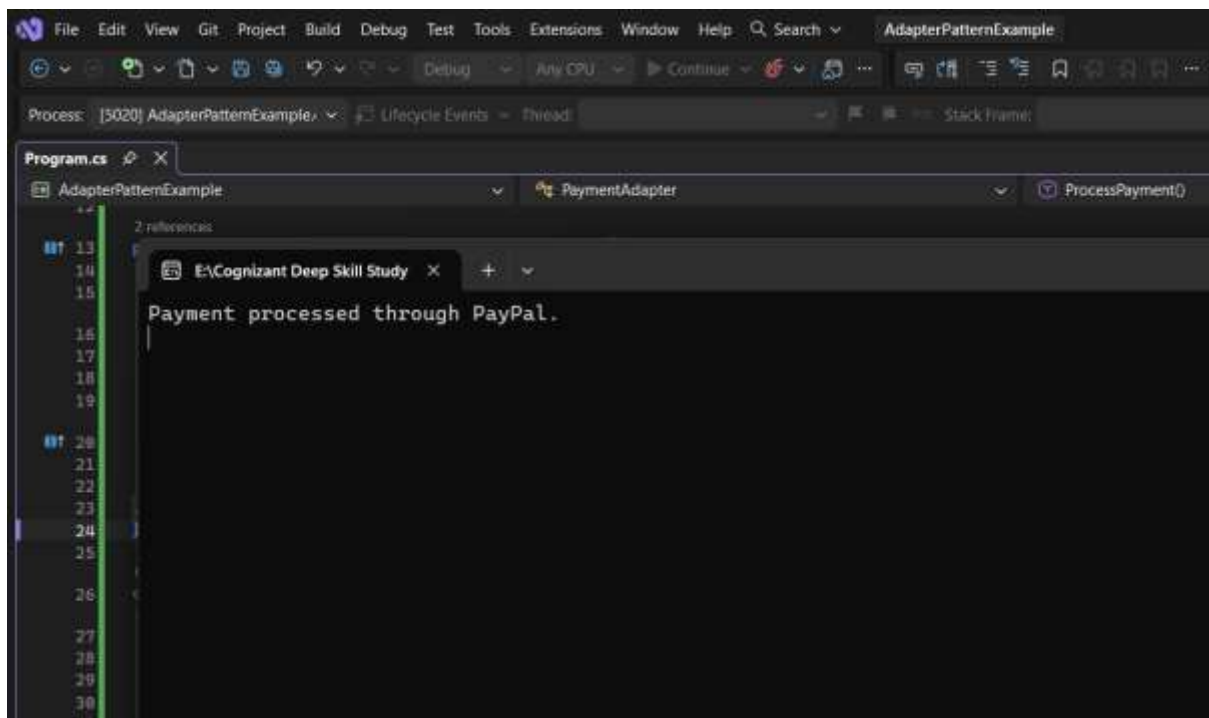


```
using System;

namespace BuilderPatternExample
{
    class Program
    {
        static void Main()
        {
            Computer Configuration :
            CPU: Intel i7
            RAM: 16 GB
            Storage: 1TB SSD

            Computer Configuration :
            CPU: Intel i5
            RAM: 8 GB
            Storage: 512GB SSD
        }
    }
}
```

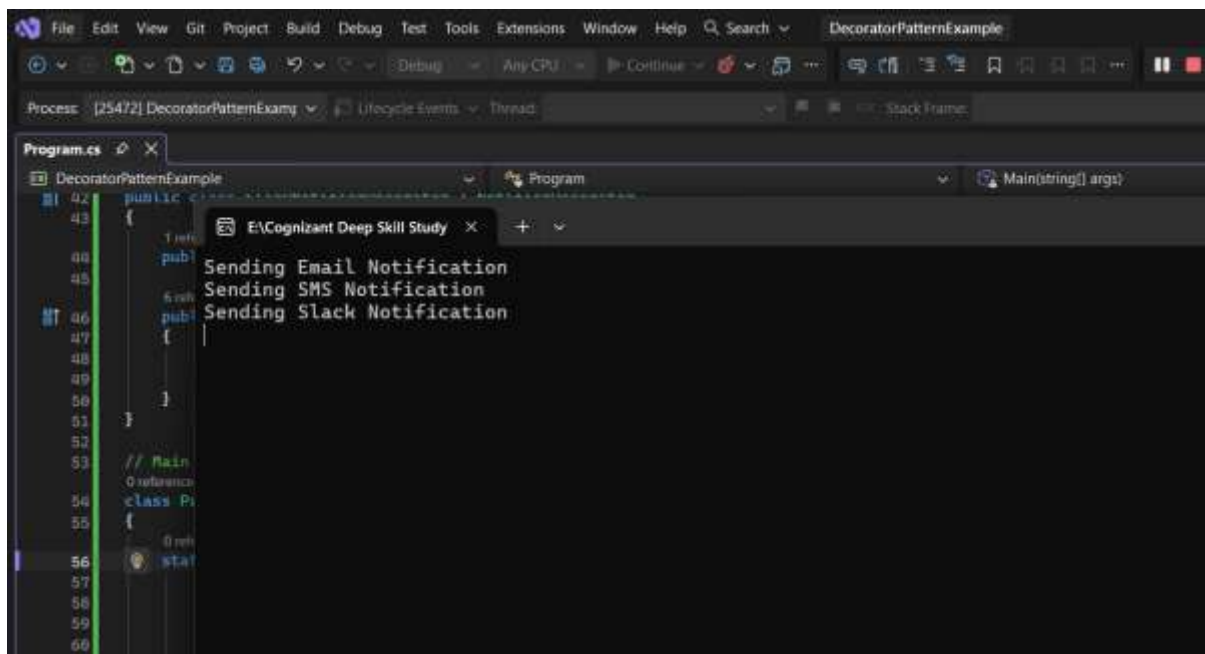
Exercise 4: Implementing the Adapter Pattern



```
2 references
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
```

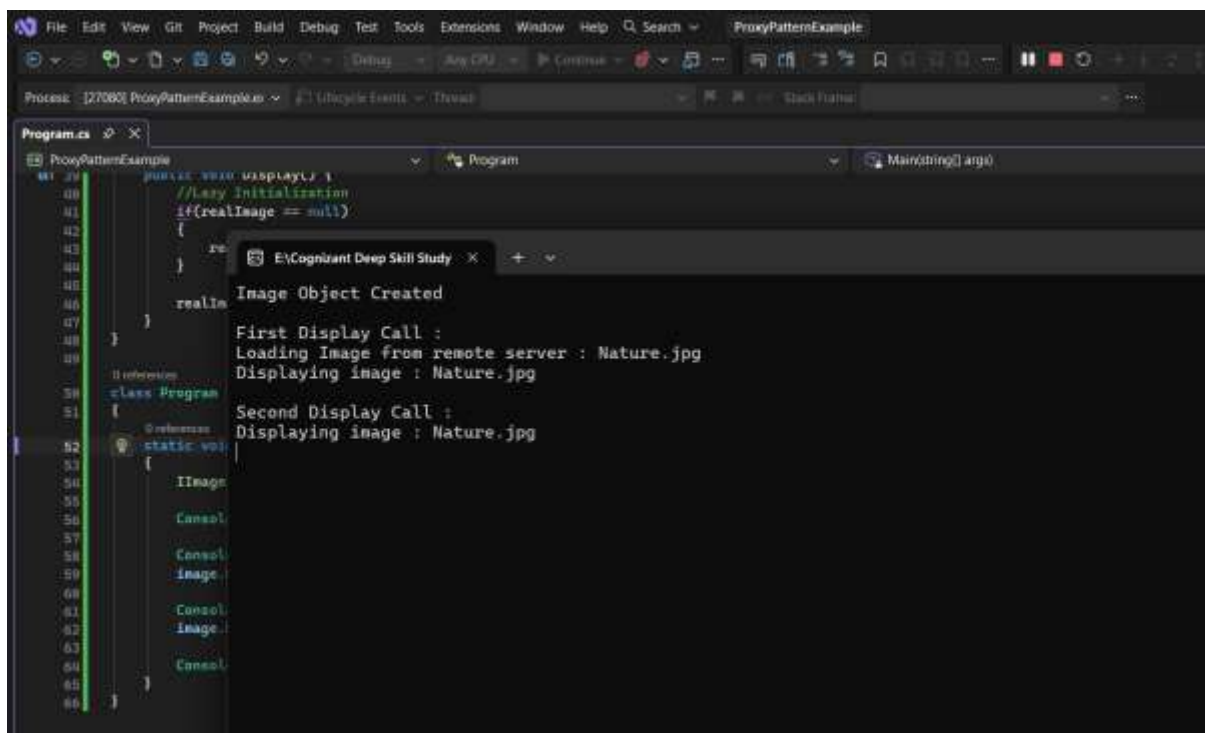
Payment processed through PayPal.

Exercise 5: Implementing the Decorator Pattern



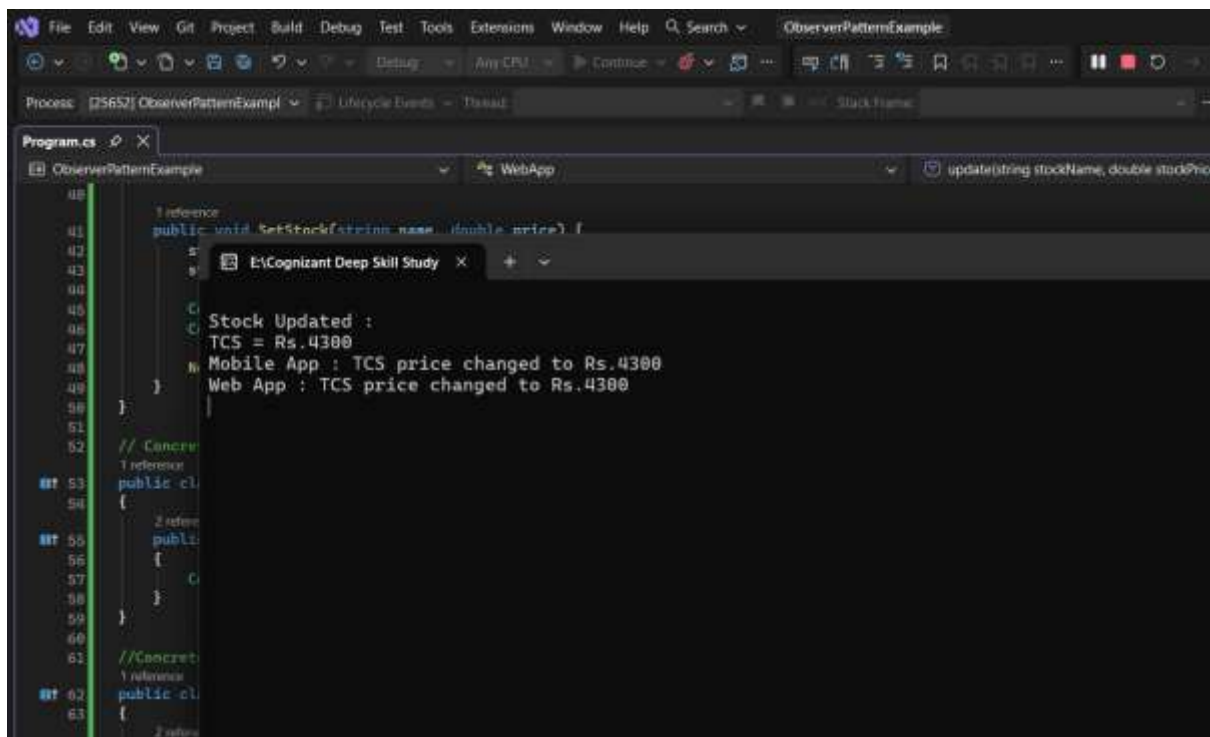
```
42 public interface IInterface
43 {
44     void Show();
45 }
46 public class EmailNotification : IInterface
47 {
48     private IInterface _wrapped;
49     public EmailNotification(IInterface wrapped)
50     {
51         _wrapped = wrapped;
52     }
53     // Main
54     public void Show()
55     {
56         Console.WriteLine("Sending Email Notification");
57         _wrapped.Show();
58     }
59 }
60 public class SMSNotification : IInterface
61 {
62     private IInterface _wrapped;
63     public SMSNotification(IInterface wrapped)
64     {
65         _wrapped = wrapped;
66     }
67     public void Show()
68     {
69         Console.WriteLine("Sending SMS Notification");
70         _wrapped.Show();
71     }
72 }
73 public class SlackNotification : IInterface
74 {
75     private IInterface _wrapped;
76     public SlackNotification(IInterface wrapped)
77     {
78         _wrapped = wrapped;
79     }
80     public void Show()
81     {
82         Console.WriteLine("Sending Slack Notification");
83         _wrapped.Show();
84     }
85 }
86 // Main
87 class Program
88 {
89     static void Main(string[] args)
90     {
91         IInterface wrapped = new EmailNotification(new SMSNotification(new SlackNotification(new IInterface() { Show: () => {} })));
92         wrapped.Show();
93     }
94 }
```

Exercise 6: Implementing the Proxy Pattern



```
29 public interface IImage
30 {
31     void Display();
32 }
33 public class Image : IImage
34 {
35     private string _url;
36     public Image(string url)
37     {
38         _url = url;
39     }
40     public void Display()
41     {
42         Console.WriteLine("Image Object Created");
43     }
44     public void LoadImage()
45     {
46         Console.WriteLine("Loading Image from remote server : Nature.jpg");
47     }
48 }
49 // Proxy
50 public class ProxyImage : IImage
51 {
52     private Image _image;
53     public ProxyImage(Image image)
54     {
55         _image = image;
56     }
57     public void Display()
58     {
59         Console.WriteLine("First Display Call :");
60         _image.LoadImage();
61         Console.WriteLine("Displaying image : Nature.jpg");
62     }
63 }
64 // Main
65 class Program
66 {
67     static void Main(string[] args)
68     {
69         Image image = new Image("Nature.jpg");
70         ProxyImage proxyImage = new ProxyImage(image);
71         proxyImage.Display();
72     }
73 }
```

Exercise 7: Implementing the Observer Pattern

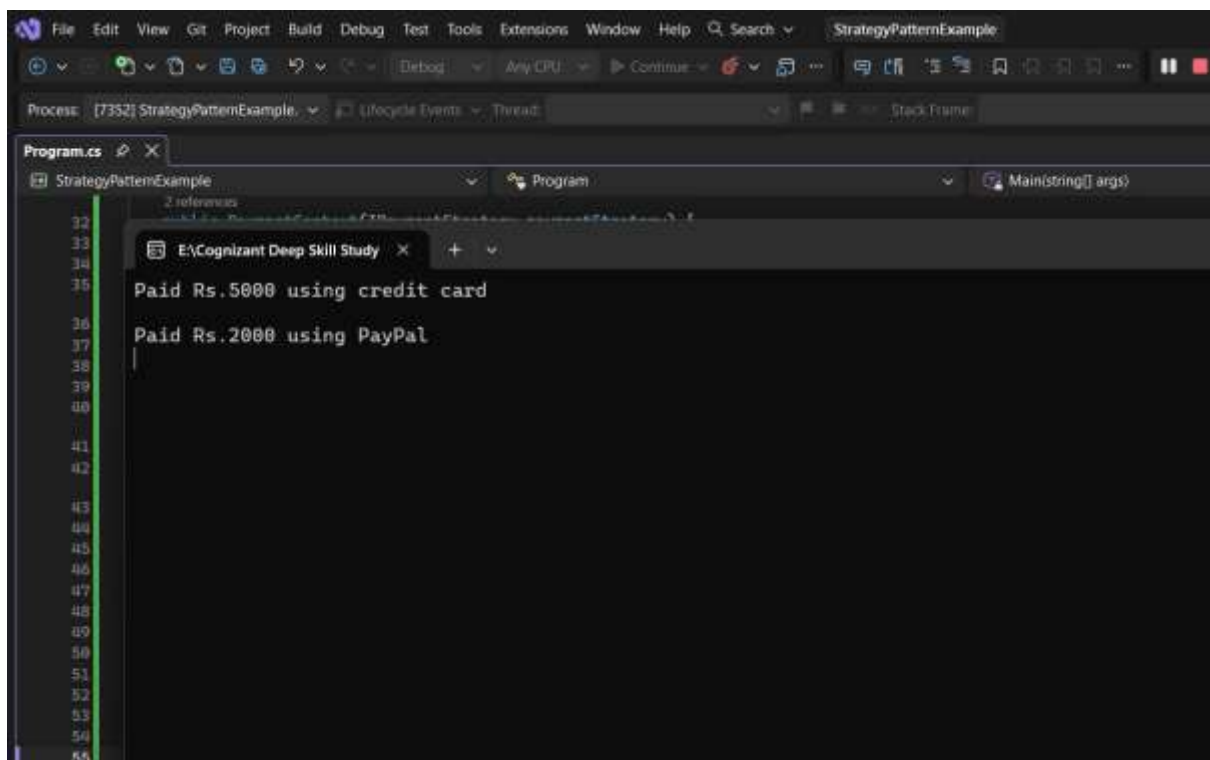


```
Program.cs
ObserverPatternExample
WebApp
update(string stockName, double stockPrice)

40
41 1 reference
42 public void SetStock(string name, double price) {
43     // ...
44     // ...
45     // ...
46     // ...
47     // ...
48     // ...
49     // ...
50 }
51
52 // Concrete
53 1 reference
54 public class ...
55 {
56     // ...
57     // ...
58     // ...
59 }
60
61 // Concrete
62 1 reference
63 public class ...
64 {
65     // ...
66     // ...
67     // ...
68 }
```

Stock Updated :
TCS = Rs.4300
Mobile App : TCS price changed to Rs.4300
Web App : TCS price changed to Rs.4300

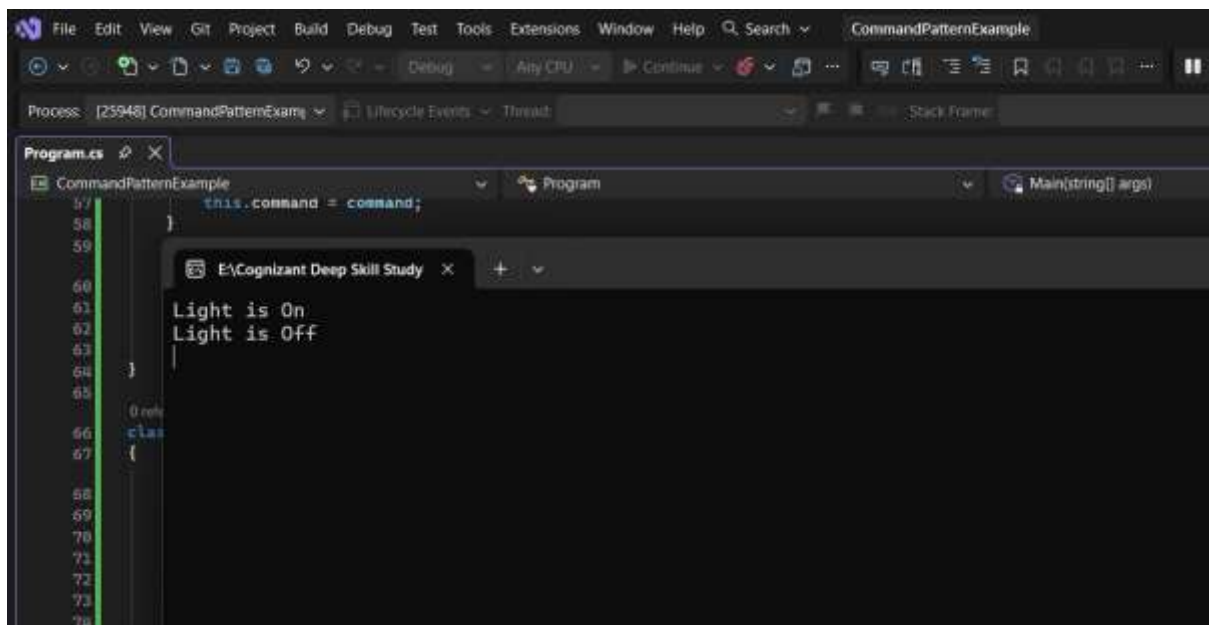
Exercise 8: Implementing the Strategy Pattern



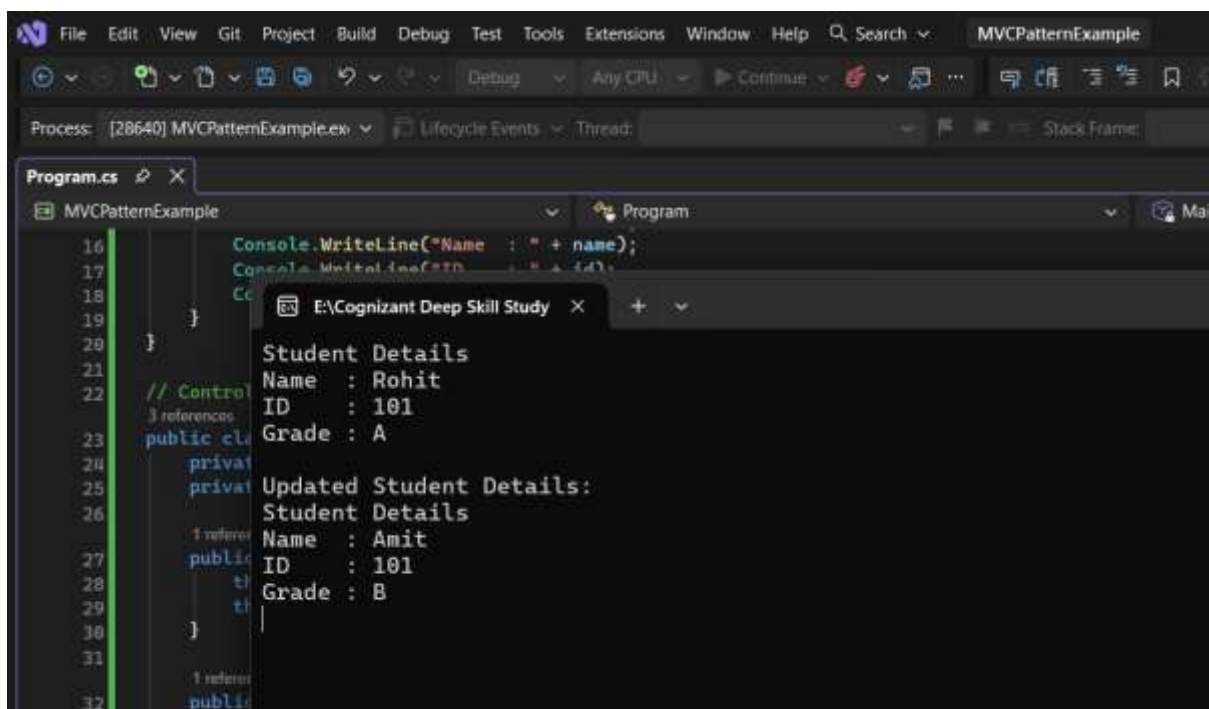
```
Program.cs
StrategyPatternExample
Program
Main(string[] args)

32
33
34
35 Paid Rs.5000 using credit card
36
37 Paid Rs.2000 using PayPal
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
```

Exercise 9: Implementing the Command Pattern



Exercise 10: Implementing the MVC Pattern



Exercise 11: Implementing Dependency Injection

